

Pointer

- C standart kütüphanesinde karakter ve metin işlemeyi sağlayan fonksiyonları inceleyeceğiz.
- Bu fonksiyonlar; metin editörleri, kelime işlemciler ve sayfa düzenleme yazılımları gibi temel yazılımların geliştirilmesinde kritik rol oynar.

Pointer

Karakterler (Characters)

- **Karakter Sabiti:** Tek tırnak içinde gösterilen bir int değeridir (Örn: 'z').
- **ASCII Değeri:** Her karakterin makine setinde bir tam sayı karşılığı vardır.
 - 'z' karakterinin değeri **122**'dir.
 - '\n' (yeni satır) karakterinin değeri **10**'dur.

Pointer

```
#include <stdio.h>
```

```
int main(void){  
    printf("'z' = %d\n", 'z');  
    printf("'\\n' = %d\n", '\\n');  
    return 0;  
}
```

ASCII'de genelde 'z'=122, '\\n'=10 olarak görülür.

Pointer

String, birden fazla karakterin tek bir birim gibi ele alınmasıdır.

Harf, rakam ve + - * / \$ gibi özel karakterler içerebilir.

C'de stringler:

- **çift tırnakla** yazılır: "blue"

- bellekte ise **char dizisi** olarak tutulur ve **sonunda mutlaka '\0' (null sonlandırıcı)** bulunur.

Yani C'de string = {'b','l','u','e','\0'} gibi bir dizidir.

Pointer

```
#include <stdio.h>

int main(void){

    char s[] = "blue"; // b l u e \0

    printf("%s\n", s);

    printf("Son karakter (null) sayisi: %d\n", s[4]); // 0

    return 0;

}
```

Pointer

Dizgiler (Strings)

C dilinde bir **dizgi (string)**, tek bir birim olarak ele alınan karakterler dizisidir.

- Yapı:** C'de stringler, sonunda **boş karakter ('\0' - null character)** bulunan karakter dizileridir.
- Erişim:** Bir stringe, stringin ilk karakterini gösteren bir **pointer (işaretçi)** aracılığıyla erişilir.
- Bir dizenin değeri, ilk karakterinin adresidir.

Pointer

```
char color[] = "blue";           // 5 elemanlı dizi: 'b', 'l', 'u', 'e', '\0'  
const char *colorPtr = "blue";  // Bellekteki "blue" dizgisini gösteren işaretçi  
char color[] = {'b', 'l', 'u', 'e', '\0'}; // Manuel tanımlama
```

```
#include <stdio.h>
```

```
int main(void){
```

```
    char a[] = "blue";
```

```
    const char *p = "blue";
```

```
    a[0] = 'B';    // OK
```

```
    printf("a = %s\n", a);
```

```
    // p[0] = 'B';    // YAPMA! (bazı sistemlerde crash)
```

```
    printf("p = %s\n", p);
```

```
    return 0;
```

```
}
```



```
#include <stdio.h>
```

```
int main(void){
```

```
    const char *p = "blue";
```

```
    printf("Tamamı: %s\n", p);
```

```
    printf("Adres : %p\n", p);
```

```
    printf("İlk chr: %c\n", *p);
```

```
    printf("1 adım : %s\n", p + 1);
```

```
    printf("1 adım : %c\n", *(p + 1));
```

```
    printf("2 adım : %s\n", p + 2);
```

```
}
```

Pointer

Sık yapılan hatalar

1.Null karakter için yer ayırmamak hata.

- "blue" 4 harf ama **5 byte** gerekir (\0 dahil).

2.Sonunda '\0' olmayan dizi string gibi yazdırmak hata.

- `printf("%s", dizi);` doğru çalışması için '\0' şart.

3.Dizi taşması (buffer overflow)

- String diziden uzun olursa, dizinin dışına yazıp başka verileri bozabilir.

Programınızda şu değişkenler **aynı isimlerle** bulunmalıdır:

- char s[] = "blue";

- char *p = s;

1) İleri yönde gezinme + küçük harfleri BÜYÜK harfe çevirme (pointer ile)

- p işaretçisi ile stringi **baştan sona** dolaşınız.

- Her karakter için:

- 1.Eğer karakter 'a' ile 'z' arasındaysa, onu **BÜYÜK harfe çeviriniz**

- İpucu: ASCII farkını kullanabilirsiniz: -32. Mevcut karakterden 32 çıkar, büyük harfe dönüştür

- 2.Her adımda şu formatta yazdırınız:

Adres: %p | Karakter: %c

- Döngünüz '\0' (string sonu) gelince bitmelidir.

2) Geri yönde gezinme (tersten yazdırma)

- İleri döngü bitince p artık '\0' karakterini gösterir.

- p'yi bir azaltarak (p--) son gerçek harfe geliniz.

- Bu kez stringi **sondan başa** doğru dolaşıp her adımda yine:

Adres: %p | Karakter: %c

formatında yazdırınız.

```

#include <stdio.h>
int main(void) {
    char s[] = "blue";
    char *p = s;
    printf("--- İleri Yonlu ve Degistirerek Gezinme ---\n");
    while (*p != '\0') {
        if (*p >= 'a' && *p <= 'z') {
            *p = *p -= 32;
        }
        printf("Adres: %p | Karakter: %c\n", p, *p);
        p++; // bir sonraki karakterin adresine gec
    }
    printf("\n--- Geri Yonlu Gezinme (Tersten Yazdirma) ---\n");
    p--; // '\0' dan onceki son harfe gel
    while (p >= s) {
        printf("Adres: %p | Karakter: %c\n", p, *p);
        p--; // bir onceki karaktere geri gel
    }
    printf("\nSon hali: %s\n", s); // BLUE
    return 0;
}

```

scanf ile String Okuma

Dizgi okurken bellek taşmalarını önlemek için mutlaka alan genişliği belirtilmelidir:

```
char word[20];
```

```
scanf("%19s", word); // En fazla 19 karakter okur, 20.  
karakter '\0' olur.
```

Pointer

Karakter İşleme Kütüphanesi (<ctype.h>)

Bu kütüphane, karakter verilerini test etmek ve manipüle etmek için kullanılan fonksiyonları içerir.

Test edilen durum doğruysa sıfırdan farklı bir değer (true), yanlışsa **0** (false) döndürür:

Prototip / Fonksiyon	Açıklama
int isblank(int c)	c, bir satırda sözcükleri ayıran boş bir karakterse doğru (true) döndürür, aksi hâlde 0 döndürür.
int isdigit(int c)	c bir rakamsa (0-9) doğru, aksi hâlde 0 döndürür.
int isalpha(int c)	c bir harfse doğru, aksi hâlde 0 döndürür.
int isalnum(int c)	c bir rakam veya harfse doğru, aksi hâlde 0 döndürür.
int isxdigit(int c)	c bir onaltılık rakamsa (A-F, a-f, 0-9) doğru, aksi hâlde 0 döndürür.
int islower(int c)	c küçük harfse (a-z) doğru, aksi hâlde 0 döndürür.
int isupper(int c)	c büyük harfse (A-Z) doğru, aksi hâlde 0 döndürür.
int tolower(int c)	c büyük harfse, küçük harf olarak döndürür. Aksi hâlde değişmeden döndürür.
int toupper(int c)	c küçük harfse, büyük harf olarak döndürür. Aksi hâlde değişmeden döndürür.
int isspace(int c)	c bir boşluk karakteriyse (\n, ' ', \f, \r, \t, \v) doğru, aksi hâlde 0 döndürür.
int iscntrl(int c)	c bir kontrol karakteriyse doğru, aksi hâlde 0 döndürür.
int ispunct(int c)	c boşluk, rakam veya harf dışında bir yazdırılabilir karakterse doğru, aksi hâlde 0 döndürür.
int isprint(int c)	c ekranda görüntülenebilen bir karakterse (boşluk dahil) doğru, aksi hâlde 0 döndürür.
int isgraph(int c)	isprint ile aynıdır; ancak boşluk karakteri dahil edilmez.

Pointer

```
#include <stdio.h>
#include <ctype.h>
```

```
int main(void){
    char c = '8';
    if (isdigit(c)) puts("digit");
    if (isalnum(c)) puts("alnum");
    return 0;
}
```


Pointer

Örnek kullanım (koşul operatörü ?:):

```
isdigit( '8' ) ? "8 bir rakamdır" : "8 bir rakam değildir"
```

- islower: Argümanın küçük harf (a-z) olup olmadığını belirler.
- isupper: Argümanın büyük harf (A-Z) olup olmadığını belirler.
- tolower: Büyük harfi küçük harfe çevirir; büyük harf değilse değişmeden döndürür.
- toupper: Küçük harfi büyük harfe çevirir; küçük harf değilse değişmeden döndürür.

Pointer

- Soru: Kullanıcıdan bir satır metin alan ve bu metin içerisindeki harf, rakam, boşluk ve noktalama işaretlerinin sayısını ayrı ayrı hesaplayıp ekrana yazdıran bir C programı yazınız.

Gereksinimler:

- Metni okumak için fgets fonksiyonunu kullanın.
- Karakter türlerini belirlemek için <ctype.h> kütüphanesindeki standart fonksiyonlardan (isalpha, isdigit vb.) faydalanın.
- Program, metnin sonundaki sonlandırıcı karakteri (\0) görene kadar tüm karakterleri tarayacaktır.

Pointer

```
#include <stdio.h>
#include <ctype.h>
```

```
int main(void){
    char line[100];
    int letter=0, digit=0, space=0, punct=0, i;

    fgets(line, 100, stdin);

    for (i=0; line[i] != '\0'; i++){
        unsigned char c = (unsigned char)line[i];
        if (isalpha(c)) letter++;
        else if (isdigit(c)) digit++;
        else if (isspace(c)) space++;
        else if (ispunct(c)) punct++;
    }

    printf("letter=%d digit=%d space=%d punct=%d\n", letter, digit, space, punct);
    return 0;
}
```

Pointer

String Dönüştürme Fonksiyonları (<stdlib.h>)

Metin (string) olarak gelen sayıları güvenli şekilde sayıya çevirir, nerede durduğunu söyler ve hatayı yakalamayı sağlar.

scanf gibi yöntemler ya satır okumada zayıf kalır ya da “ne kadarını çevirdi / hata oldu mu?” bilgisini vermez. Bu fonksiyonlar ise tam tersine kontrollü dönüşüm sağlar.

Pointer

String Dönüştürme Fonksiyonları (<stdlib.h>)

Bir formdan "51.2% are admitted" gibi bir ifade gelebilir.

- İhtiyaç: 51.2 sayısını almak
- Kalan metni: "% are admitted" şeklinde elde etmek

Bunu `scanf("%lf")` ile her zaman düzgün yönetemezsin;

Bazen veri size bütün bir cümle olarak gelir, içindeki sayıyı almamız gerekir.

Pointer

String Dönüştürme Fonksiyonları (<stdlib.h>)

1. strtod (String to Double)

Bir stringi double türüne dönüştürür.

```
char *metin = "51.2% oraninda";
```

```
char *kalan;
```

```
double d = strtod(metin, &kalan);
```

```
// d = 51.20
```

```
// kalan = "% oraninda"
```

String Dönüştürme Fonksiyonları (<stdlib.h>)

2. strtol (String to Long)

Bir stringi belirtilen tabanda (base) long int değerine dönüştürür.

```
char *metin = "-1234abc";
```

```
char *kalan;
```

```
long x = strtol(metin, &kalan, 10);
```

```
// x = -1234
```

```
// kalan = "abc"
```

Pointer

Standart Giriş/Çıkış Kütüphanesi Fonksiyonları

puts ne işe yarar?

- Ekran (standart çıktı) **bir string** yazdırır.
- Yazdırdıktan sonra **otomatik olarak \n (yeni satır)** ekler.

Örnek:

```
puts("Merhaba"); // ekrana "Merhaba\n" yazar
```

gets ne işe yarar?

- Klavyeden (standart girdi) **bir satır** okumaya çalışır ve okuduğunu bir char[] içine yazar.
- Kullanıcı Enter'a basana kadar okur, sonuna '\0' koyar.

Örnek (ama KULLANMAYIN):

```
gets(buf); // buf içine satırı yazar (tehlikeli)
```

Kritik not: gets **kaç karakter okunacağını bilmez**, bu yüzden dizi taşmasına (buffer overflow) yol açabilir. Bu sebeple modern C'de **kullanılması önerilmez**, hatta standarttan kaldırılmıştır.

Standart Giriş/Çıkış Kütüphanesi Fonksiyonları

fgets neden gets yerine?

- **gets güvensizdir:** kaç karakter girileceğini sınırlandıramaz → **buffer overflow** (taşma) riski. Bu yüzden C standardından kaldırıldı.
- **fgets güvenlidir:** kaç karakter okunacağını (n-1) sen belirlersin, dizi taşmasını büyük ölçüde önler. Ayrıca stdin veya dosyadan da okuyabilir.

fputs neden puts yerine?

- **puts otomatik \n ekler:** her zaman satır sonu istersen güzel, ama istemezsen kontrol sende değildir.
- **fputs \n eklemez:** satır sonunu **sen yönetirsin** ("\n" basarsın ya da basmazsın). Ayrıca stdout dışında **dosyaya** da yazabilir.

Standart Giriş/Çıkış Kütüphane Fonksiyonları

fgets — satır okuma (güvenli)

fgets fonksiyonu belirtilen bir dosyadan (veya stdin'den) tek satır okur. En fazla n-1 karakter okur, \0 koyar; genelde stdin ile kullanılır.

```
#include <stdio.h>
```

```
int main(void){
```

```
char line[20];
```

```
if (fgets(line, 20, stdin) != NULL) {
```

```
printf("line=%s", line);
```

```
}
```

```
return 0;
```

```
}
```

Pointer

Standart Giriş/Çıkış Kütüphanesi Fonksiyonları

fputs — string yazma (newline eklemez)

```
#include <stdio.h>

int main(void){

fputs("Hello", stdout);

fputs("\n", stdout);

return 0;

}
```

Standart Giriş/Çıkış Kütüphane Fonksiyonları

getchar — tek karakter okuma

```
#include <stdio.h>

int main(void){

int c = getchar();

printf("okunan=%c (kod=%d)\n", c, c);

return 0;

}
```

Standart Giriş/Çıkış Kütüphane Fonksiyonları

putchar — tek karakter yazma

```
#include <stdio.h>

int main(void){

    putchar('A');

    putchar('\n');

    return 0;

}
```

Pointer

Standart Giriş/Çıkış Kütüphane Fonksiyonları

sprintf Fonksiyonu

sprintf, printf ile aynı şekilde çalışır, ancak çıktıyı ekrana değil bir dizeye (karakter dizisine) yazar.

```
sprintf( buffer, "%d-%d-%d", gun, ay, yil );
```

```
#include <stdio.h>
int main(void){
    char buf[30];
    int n = 7;
    printf("n=%d\n", n);
    sprintf(buf, "n=%d", n);
    puts(buf);
    return 0;
```

Standart Giriş/Çıkış Kütüphanesi Fonksiyonları

sscanf Fonksiyonu

sscanf, scanf ile aynı şekilde çalışır, ancak girdiyi klavyeden değil bir dizeden okur.

```
sscanf( dize, "%s%d", isim, &yas );
```

```
#include <stdio.h>
int main(void){
    const char *s = "ID=12 TEMP=36";
    int id, temp;
    sscanf(s, "ID=%d TEMP=%d", &id, &temp);
    printf("id=%d temp=%d\n", id, temp);
    return 0;
}
```

```
#include <stdio.h>

int main(void) {
    // Verilerimiz (Şimdilik değişkenlerde)
    char istasyon[] = "ANKARA_METEO";
    float sicaklik = 22.5;
    int nem = 45;
    char paket[100]; // Veriyi saklayacağımız "kutu"
    // 1. ADIM: PAKETLEME (sprintf)
    // Verileri tek bir metin haline getirip 'paket' içine koyuyoruz.
    sprintf(paket, "%s %.1f %d", istasyon, sicaklik, nem);
    printf("Paketlenmis veri: %s\n", paket);
    printf("--- Veri su an bellekte (paket dizisinde) saklanıyor ---\n\n");
    // ... Aradan zaman geçtiğini veya bu metnin başka bir fonksiyona gittiğini düşün ...
    // 2. ADIM: PAKETİ AÇMA (sscanf)
    // Şimdi 'paket' içindeki o tek satır metni geri değişkenlere parçalayalım.
    char okunan_istasyon[50];
    float okunan_sicaklik;
    int okunan_nem;
    sscanf(paket, "%s %f %d", okunan_istasyon, &okunan_sicaklik, &okunan_nem);
    // Artık bu verilerle matematiksel işlem yapabiliriz!
    printf("Paketten cikan sicaklik: %.1f derece\n", okunan_sicaklik);
    printf("Nem oranı: %%%d\n", okunan_nem);
    return 0;
}
```


Fonksiyon	Kısa açıklama	Neden kullanılır?	Kısa örnek kullanım
getchar	Klavyeden 1 karakter okur (int döner).	“Devam için tuş bekle”, karakter karakter okuma/parse etme.	int c = getchar();
putchar	Ekrana 1 karakter yazar.	Karakter karakter yazdırma, filtreleme (sadece harfleri bas vb.).	putchar('A'); putchar('\n');
puts	Stringi yazar ve sonuna otomatik \n ekler.	Hızlı mesaj/satır yazdırma.	puts("Merhaba dünya");
fgets	stdin'den satır okur (en fazla n-1), '\0' koyar.	Boşluklu cümle okumak ve taşmayı azaltmak.	fgets(line, sizeof(line), stdin);
fputs	Stringi yazar, \n eklemez .	Newline kontrolü sende olsun; dosyaya/ekrana düz yazı bas.	fputs("Hello", stdout);
sprintf	Formatlı çıktıyı stringe yazar (taşma riski).	“Mesaj üretme”: log satırı, birleşik çıktı hazırlama.	sprintf(buf, "ID=%d", id);
sscanf	Bir stringin içinden formatla okur (parse eder).	Log/komut gibi metinden sayıları çekmek.	sscanf(s, "ID=%d TEMP=%d", &id, &temp);

```
#include <stdio.h>

int main(void) {
    char line[100], name[50], info[100];
    int age;
    char confirm;
    puts("--- Kullanici Kayit Sistemine Hos Geldiniz ---"); //1
    fputs("Kayit baslatilsin mi? (e/h): ", stdout); //2
    confirm = getchar(); //3
    fputs("Sectiginiz onay: ", stdout); //4
    putchar(confirm);
    putchar('\n'); // Alt satıra geçmek için
    while (getchar() != '\n');
    if (confirm == 'e' || confirm == 'E') {
        puts("\nLutfen adinizi ve yasinizi boslukla ayirarak girin (Ornek: Can 25):");
        fgets(line, sizeof(line), stdin); //5
        if (sscanf(line, "%s %d", name, &age) == 2) { //6
            sprintf(info, "Kayit Ozeti: Isim: %s | Yas: %d", name, age); //7
            puts(info);
        } else {
            puts("Hatali giris yaptiniz!");
        }
    } else {
        puts("Islem iptal edildi.");
    }
    return 0;
```

Pointer

- 1.Kullanıcıya "**Bir satir giriniz:**" mesajını puts ile yazdırınız.
- 2.Kullanıcının girdiği satırı **fgets** ile line adlı char dizisine okuyunuz. (line boyutu en az 50 olsun.)
- 3.Okunan satırı **printf** ile şu formatta ekrana yazdırınız:
 - Girdiginiz satir: <satir>
- 4.Kullanıcıdan devam etmek için bir tuşa basmasını isteyiniz:
 - "**Devam etmek icin bir tusa basin:**" mesajını puts ile yazdırınız.
 - Ardından **getchar()** çağırarak bir karakter okuyunuz (devam için).
- 5.Son olarak satırdaki karakterleri **putchar** kullanarak **tek tek** tekrar ekrana yazdırınız.
 - Döngü, '\\0' karakterini görene kadar sürmelidir.

Not: Stringi yazdırırken %s, karakterleri yazdırırken putchar kullanınız.

Pointer

```
#include <stdio.h>
```

```
int main(void){  
    char line[50];
```

```
    puts("Bir satir giriniz:");  
    if (fgets(line, sizeof(line), stdin) == NULL) return 0;
```

```
    printf("Girdiginiz satir: %s", line);
```

```
    puts("\nDevam etmek icin bir tusa basin:");  
    getchar(); // Enter kalabilir; derste bunun etkisini anlatabilirsin
```

```
    puts("Satirdaki karakterleri tek tek yazdiriyorum:");  
    for (int i = 0; line[i] != '\0'; i++){  
        putchar(line[i]);  
    }
```

```
    return 0;
```

```
}
```

Pointer

Dize işleme kütüphanesi (<string.h>), dize manipülasyonu için çok sayıda fonksiyon içerir.

strcpy ve strncpy Fonksiyonları

- `strcpy(s1, s2)`: s2'yi s1'e kopyalar (null sonlandırıcı dahil).
- `strncpy(s1, s2, n)`: s2'den en fazla n karakter s1'e kopyalar.

```
char dize1[10]; strcpy( dize1, "Merhaba" ); strncpy( dize1, "Merhaba", 3 ); // dize1 = "Mer"
```

Pointer

strcat ve strncat Fonksiyonları

- `strcat(s1, s2)`: `s2`'yi `s1`'in sonuna ekler.
- `strncat(s1, s2, n)`: `s2`'den en fazla `n` karakteri `s1`'in sonuna ekler.

```
char dize1[20] = "Merhaba "; strcat( dize1, "Dünya" ); // dize1 =  
"Merhaba Dünya"
```

Pointer

Karşılaştırma Fonksiyonları

strcmp(s1, s2) ve strncmp(s1, s2, n) fonksiyonları iki dizeyi karşılaştırır:

- $s1 < s2$ ise negatif bir değer döndürür.
- $s1 == s2$ ise 0 döndürür.
- $s1 > s2$ ise pozitif bir değer döndürür.

```
if ( strcmp( dize1, dize2 ) == 0 )    printf( "Dizeler eşittir.\n" );
```